

Отчёт по лабораторной работе №7

Компьютерный практикум по статистическому анализу данных

Канева Екатерина, НФИбд-02-22

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическая часть	7
4	Выполнение лабораторной работы	8
4.1	Примеры	8
4.2	Задания для самостоятельной работы	12
4.2.1	Задание 1	12
4.2.2	Задание 2	17
4.2.3	Задание 3	19
5	Выводы	22

Список иллюстраций

4.1	Примеры со считыванием данных.	8
4.2	Примеры со словарями.	9
4.3	Примеры с missing типом.	9
4.4	Примеры с join.	9
4.5	Пример со считыванием картинки.	10
4.6	Примеры с кластеризацией, метод k средних.	11
4.7	Пример с линейной регрессией.	12
4.8	Загрузка, кластеризация данных.	13
4.9	Визуализация данных.	14
4.10	Добавление центроидов.	15
4.11	Создание датафрейма с результатами кластеризации.	16
4.12	Анализ качества.	17
4.13	Информация о кластерах.	17
4.14	Генерация данных и ручная оценка.	18
4.15	Сопоставление результатов.	18
4.16	Линейная регрессия и график.	19
4.17	Модель ценообразования биномиальных опционов.	20
4.18	График траекторий цены акции.	20
4.19	График для различных траекторий цены.	21
4.20	Параллельное вычисление траекторий.	21

Список таблиц

1 Цель работы

Основной целью работы является освоение специализированных пакетов Julia для обработки данных.

2 Задание

- Используя Jupyter Lab, повторить примеры.
- Выполнить задания для самостоятельной работы.

3 Теоретическая часть

Julia - высокоуровневый свободный язык программирования с динамической типизацией, созданный для математических вычислений. Эффективен также и для написания программ общего назначения. Синтаксис языка схож с синтаксисом других математических языков, однако имеет некоторые существенные отличия.

Для выполнения заданий была использована официальная документация Julia.

4 Выполнение лабораторной работы

4.1 Примеры

Сначала я выполнила примеры из лабораторной работы (рис. 4.1-4.7):

```
[7]: function language_created_year(P,language::String)
      loc = findfirst(P[:,2].==language)
      return P[loc,1]
      end

[7]: language_created_year (generic function with 1 method)

[9]: # Пример вызова функции и определение даты создания языка Python:
      language_created_year(P,"Python")

[9]: 1991

[11]: # Пример вызова функции и определение даты создания языка Julia:
      language_created_year(P,"Julia")

[11]: 2012

[15]: # Функция определения по названию языка программирования
      # года его создания (без учёта регистра):
      function language_created_year_v2(P,language::String)
          loc = findfirst(lowercase.(P[:,2]).==lowercase.(language))
          return P[loc,1]
          end

[15]: language_created_year_v2 (generic function with 1 method)

[19]: language_created_year_v2(P,"julia")

[19]: 2012
```

Рис. 4.1: Примеры со считыванием данных.


```
[21]: # Запись данных в CSV-файл:
      CSV.write("programming_languages_data2.csv", P)

[21]: "programming_languages_data2.csv"

[27]: # Инициализация словаря:
      dict = Dict{Integer,Vector{String}}{()}

[27]: Dict{Integer, Vector{String}}{()}

[29]: # Заполнение словаря данными:
      for i = 1:size(P,1)
          year,lang = P[i,:]
          if year in keys(dict)
              dict[year] = push!(dict[year],lang)
          else
              dict[year] = [lang]
          end
      end

[31]: dict[2003]

[31]: 2-element Vector{String}:
      "Groovy"
      "Scala"
```

Рис. 4.2: Примеры со словарями.

```
[39]: # Определения типа переменной:
      typeof(iris)

[39]: DataFrame

[41]: describe(iris)

[41]: 5×7 DataFrame
      Row  variable    mean    min    median    max    nmissing  eltype
      ───  ────
      1  SepalLength  5.84333  4.3    5.8      7.9      0      Float64
      2  SepalWidth   3.05733  2.0    3.0      4.4      0      Float64
      3  PetalLength   3.758    1.0    4.35     6.9      0      Float64
      4  PetalWidth    1.19933  0.1    1.3      2.5      0      Float64
      5  Species              setosa      virginica      0      CategoricalValue{String, UInt8}

[43]: # Отсутствующий тип:
      a = missing
      typeof(a)

[43]: Missing

[45]: a + 1

[45]: missing
```

Рис. 4.3: Примеры с missing типом.

```
[59]: # Задание сведений о ценах:
      prices = [0.85,1.6,0.8,0.6]
      # Формирование данных о калориях:
      dataframe_calories = DataFrame(item=foods,calories=calories)
      # Формирование данных о ценах:
      dataframe_prices = DataFrame(item=foods,price=prices)
      # Объединение данных о калориях и ценах:
      DF = innerjoin(dataframe_calories,dataframe_prices,on=:item)

[59]: 4×3 DataFrame
      Row  item    calories  price
      ───  ────
      1  apple    missing    0.85
      2  cucumber  47        1.6
      3  tomato    22        0.8
      4  banana    105       0.6
```

Рис. 4.4: Примеры с join.

```
[69]: # Загрузка изображения:
X1 = load("julialogo.png")

[69]:
```



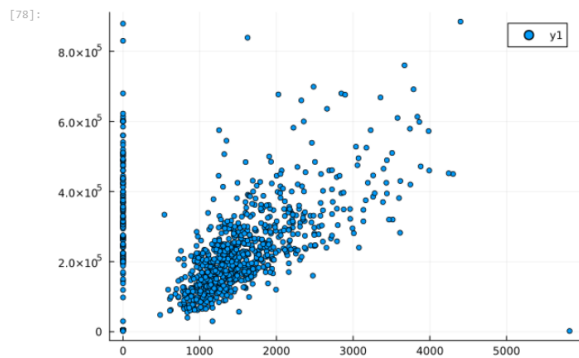
The image shows the Julia logo, which consists of the word "julia" in a bold, black, sans-serif font. Above the letters "i", "l", and "a" are four colored circles: a blue circle above the "i", a red circle above the "l", and a green circle above the "a". A purple circle is positioned between the red and green circles, slightly above the "i".

```
[71]: # Определение типа и размера данных:
@show typeof(X1);
@show size(X1);

typeof(X1) = IndirectArrays.IndirectArray{ColorTypes.RGB{FixedPointNumbers.N0f8}, 2, UInt8, Matrix{UInt8}, OffsetArrays.OffsetVector{ColorTypes.RGB{FixedPointNumbers.N0f8}}, Vector{ColorTypes.RGB{FixedPointNumbers.N0f8}}}}
size(X1) = (180, 280)
```

Рис. 4.5: Пример со считыванием картинки.

```
[78]: # Построение графика:
using Plots
plot(size=(500,500),leg=false)
x = houses[:,sq_ft]
y = houses[:,price]
scatter(x,y,markersize=3)
```



```
[80]: # Фильтрация данных по заданному условию:
filter_houses = houses[houses[:,sq_ft].>0,:]
# Построение графика:
x = filter_houses[:,sq_ft]
y = filter_houses[:,price]
scatter(x,y)
```

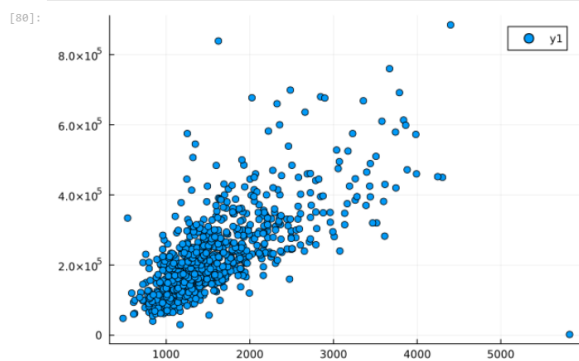


Рис. 4.6: Примеры с кластеризацией, метод к средних.

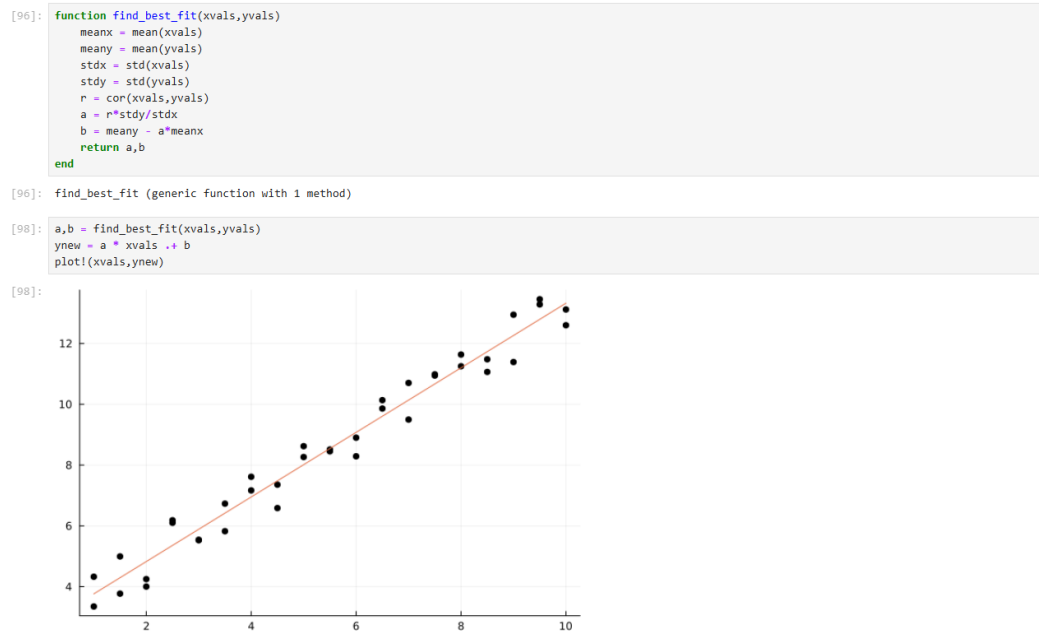


Рис. 4.7: Пример с линейной регрессией.

4.2 Задания для самостоятельной работы

Далее я приступила к выполнению заданий для самостоятельной работы.

4.2.1 Задание 1

Загрузим

```
using RDatasets
```

```
iris = dataset("datasets", "iris")
```

Используем Clustering.jl для кластеризации на основе k-средних. Сделаем точечную диаграмму полученных кластеров. Для этого проиндексируем фрейм данных, преобразуем его в массив и транспонируем.

Сначала загрузила данные и кластеризовала (рис. 4.8):

```
[116]: using RDatasets
iris = dataset("datasets", "iris")
```

[116]: 150x5 DataFrame 125 rows omitted

Row	SepalLength	SepalWidth	PetalLength	PetalWidth	Species
	Float64	Float64	Float64	Float64	Cat...
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa
7	4.6	3.4	1.4	0.3	setosa

```
[120]: iris_features = iris[:, 1:4]
iris_matrix = Matrix(iris_features)
println("Размерность матрицы признаков: ", size(iris_matrix))
Размерность матрицы признаков: (4, 150)
```

```
[130]: k = 3
result = kmeans(iris_matrix, k)

cluster_labels = assignments(result)

centroids = result.centers

println("Количество кластеров: ", k)
println("Размеры кластеров: ", counts(result))
println("Итераций: ", result.iterations)
println("Общая стоимость (сумма квадратов расстояний): ", round(result.totalcost, digits=2))

Количество кластеров: 3
Размеры кластеров: [50, 61, 39]
Итераций: 2
Общая стоимость (сумма квадратов расстояний): 78.86
```

Рис. 4.8: Загрузка, кластеризация данных.

Визуализировала кластеры (рис. 4.9):

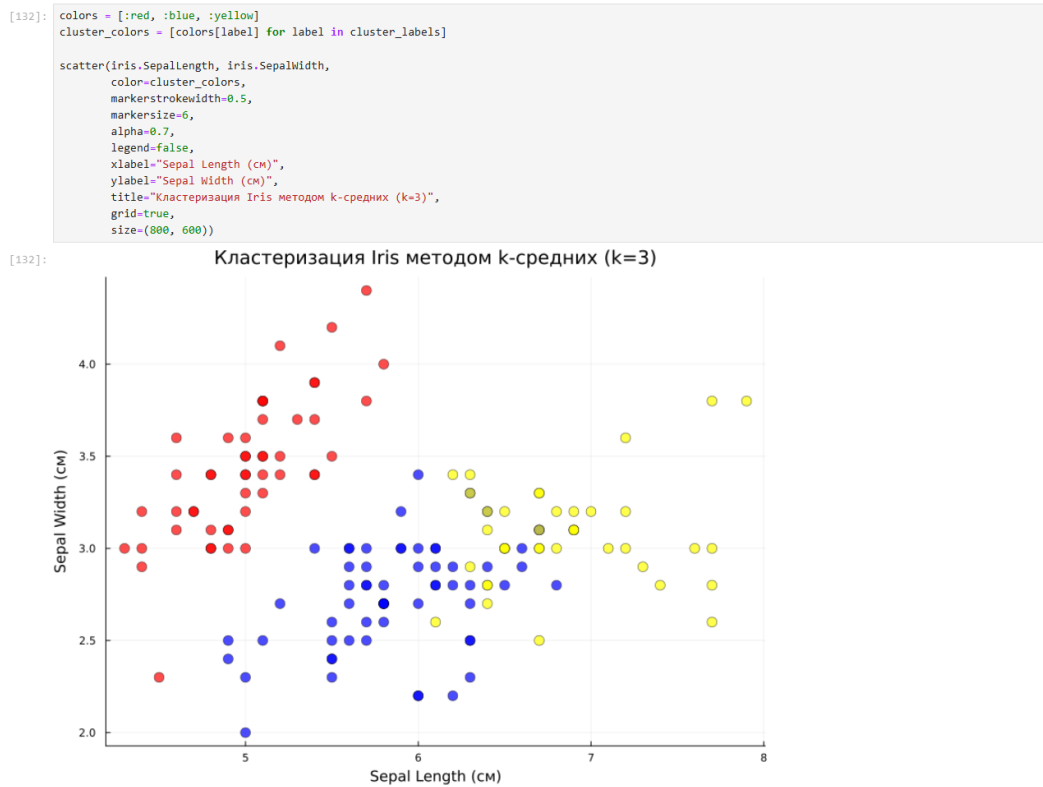


Рис. 4.9: Визуализация данных.

Добавила центры (рис. 4.10):

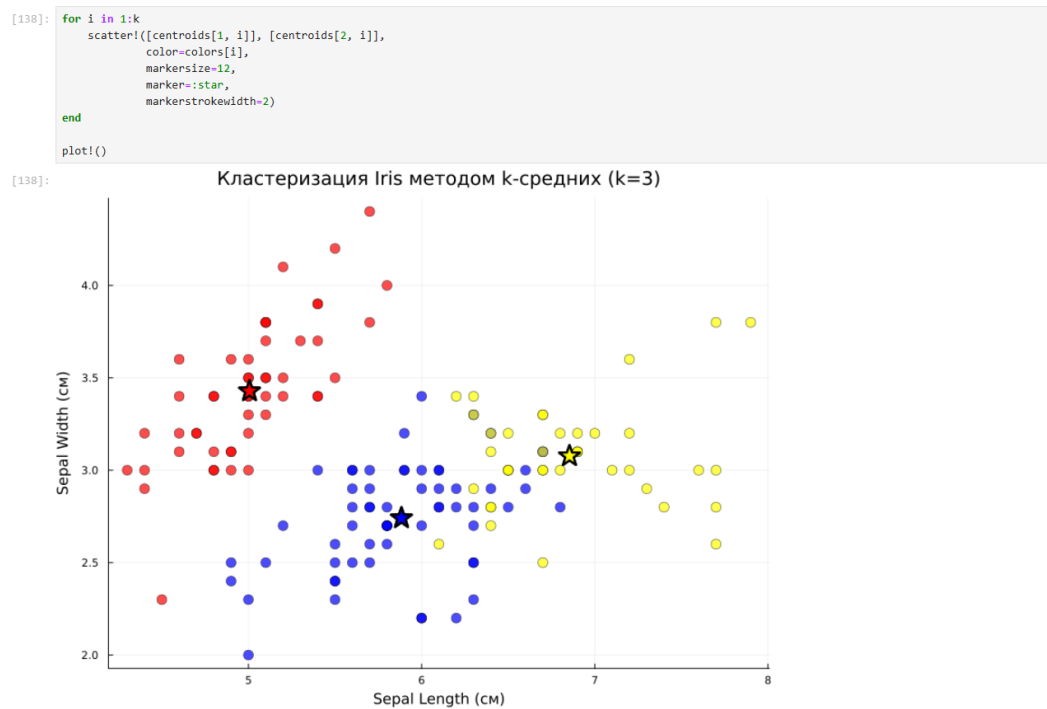


Рис. 4.10: Добавление центроидов.

Создала датафрейм с результатами кластеризации (рис. 4.11):

```
[148]: # Создаем DataFrame с результатами кластеризации
using DataFrames
iris_with_clusters = copy(iris)
iris_with_clusters[:, :Cluster] = cluster_labels

# Функция для создания матрицы парных диаграмм
function pairplot_matrix(df)
    features = ["SepalLength", "SepalWidth", "PetalLength", "PetalWidth"]
    n = length(features)

    plot_array = []

    for i in 1:n
        for j in 1:n
            if i == j
                # Диагональ - гистограмма распределения
                p = histogram(df[:, features[i]],
                    color=:gray, alpha=0.7,
                    legend=false,
                    title=features[i],
                    grid=false)
            else
                # Внедиагональные элементы - scatter plots
                p = scatter(df[:, features[j]], df[:, features[i]],
                    color=cluster_colors,
                    markerstrokewidth=0,
                    markersize=4,
                    alpha=0.6,
                    legend=false,
                    grid=false)
            end
            push!(plot_array, p)
        end
    end

    return plot(plot_array..., layout=(n, n), size=(1000, 1000))
end

# Создаем и сохраняем матрицу парных диаграмм
pairplot = pairplot_matrix(iris_with_clusters)
savefig(pairplot, "iris_pairplot_matrix.png")
println("Матрица парных диаграмм сохранена как 'iris_pairplot_matrix.png'")

# Сравнение кластеров с реальными видами ирисов
println("\nСравнение кластеров с реальными видами:")

# Создаем таблицу сопряженности
real_species = iris.Species
cluster_df = DataFrame(RealSpecies=real_species, Cluster=cluster_labels)

Матрица парных диаграмм сохранена как 'iris_pairplot_matrix.png'
```

Рис. 4.11: Создание датафрейма с результатами кластеризации.

Сделала анализ качества кластеризации (рис. 4.12):


```
[142]: # Отобразим распределение видов по кластерам
for species in unique(real_species)
  println("\nВид: $species")
  species_clusters = cluster_df[cluster_df.RealSpecies .== species, :Cluster]
  for cl in 1:k
    count = sum(species_clusters .== cl)
    percentage = round(100 * count / length(species_clusters), digits=1)
    println("    Кластер $cl: $count образцов ($percentage%)")
  end
end

# Анализ качества кластеризации
println("\n\nАнализ качества кластеризации:")
println("Центроиды кластеров:")
for i in 1:k
  println("Кластер $i: ", round.(centroids[:, i], digits=2))
end

Вид: setosa
  Кластер 1: 50 образцов (100.0%)
  Кластер 2: 0 образцов (0.0%)
  Кластер 3: 0 образцов (0.0%)

Вид: versicolor
  Кластер 1: 0 образцов (0.0%)
  Кластер 2: 47 образцов (94.0%)
  Кластер 3: 3 образцов (6.0%)

Вид: virginica
  Кластер 1: 0 образцов (0.0%)
  Кластер 2: 14 образцов (28.0%)
  Кластер 3: 36 образцов (72.0%)

Анализ качества кластеризации:
Центроиды кластеров:
Кластер 1: [5.01, 3.43, 1.46, 0.25]
Кластер 2: [5.88, 2.74, 4.39, 1.43]
Кластер 3: [6.85, 3.08, 5.72, 2.05]
```

Рис. 4.12: Анализ качества.

Вывела итоговую сводную информацию о кластерах (рис. 4.13):

```
[144]: # Расчет инерции (суммы квадратов расстояний до центроидов)
println("\n\nИнформация о кластерах:")
for i in 1:k
  cluster_points = iris_matrix[:, cluster_labels .== i]
  cluster_size = size(cluster_points, 2)

  if cluster_size > 0
    # Среднее расстояние до центроида
    distances = [sqrt(sum((cluster_points[:, j] - centroids[:, i]).^2))
                 for j in 1:cluster_size]
    avg_distance = round(mean(distances), digits=2)
    std_distance = round(std(distances), digits=2)

    println("Кластер $i: $cluster_size образцов, среднее расстояние до центроида: $avg_distance ± $std_distance")
  end
end

Информация о кластерах:
Кластер 1: 50 образцов, среднее расстояние до центроида: 0.48 ± 0.27
Кластер 2: 61 образцов, среднее расстояние до центроида: 0.73 ± 0.31
Кластер 3: 39 образцов, среднее расстояние до центроида: 0.73 ± 0.35
```

Рис. 4.13: Информация о кластерах.

4.2.2 Задание 2

Пусть регрессионная зависимость является линейной. Матрица наблюдений факторов X имеет размерность $N \times 3$ ($N, 3$), массив результатов в $N \times 1$, регрессионная зависимость является линейной. Найдем МНК-оценку для линейной модели:

- Сравним свои результаты с результатами использования `llsq` из `MultivariateStats.jl`.
- Сравним свои результаты с результатами использования регулярной регрессии наименьших квадратов из `GLM.jl`.

Сгенерировала данные и сделала ручную оценку (рис. 4.14):

```
[151]: # Генерация данных
N = 1000
X = randn(N, 3)
a0 = rand(3)
y = X * a0 + 0.1 * randn(N)

# МНК-оценка вручную (с добавлением столбца единиц для свободного члена)
X2 = hcat(ones(N), X)
a_mnk = (X2' * X2) \ (X2' * y) # Решение системы (X'X)a = X'y
println("МНК-оценка (вручную): ", a_mnk)

# Сравнение с MultivariateStats.jl (функция `llsq`)
a_llsq = llsq(X, y, bias=true) # bias=true добавляет константу (столбец единиц)
println("МНК-оценка (MultivariateStats.jl, llsq): ", a_llsq)

МНК-оценка (вручную): [-0.0014444924711317668, 0.8776963668618514, 0.6533362985380224, 0.5258295402822943]
МНК-оценка (MultivariateStats.jl, llsq): [0.8776963668618513, 0.6533362985380224, 0.5258295402822944, -0.0014444924711317733]
```

Рис. 4.14: Генерация данных и ручная оценка.

Сопоставила результаты (рис. 4.15):

```
[163]: # Сравнение с GLM.jl (регулярная линейная регрессия)
# Создаем DataFrame для GLM (если не установлен – установите пакет DataFrames)
using DataFrames
df = DataFrame(X1=X[:,1], X2=X[:,2], X3=X[:,3], y=y)
model_glm = lm(@formula(y ~ X1 + X2 + X3), df)
a_glm = coef(model_glm)
println("МНК-оценка (GLM.jl): ", a_glm)

# Проверка совпадения результатов
println("\nСравнение результатов:")
println("Разница между ручным МНК и llsq: ", round(norm(a_mnk - a_llsq), digits=3))
println("Разница между ручным МНК и GLM: ", round(norm(a_mnk - a_glm), digits=3))

# Теоретическое объяснение
println("""
Матрица X2 = [1 X] позволяет оценить модель y = β₀ + β₁x₁ + β₂x₂ + β₃x₃.
Теоретически, МНК-оценка находится как a = (X2'X2)⁻¹X2'y.
Это стандартный метод решения overdetermined системы уравнений.
""")

МНК-оценка (GLM.jl): [-0.0014444924711317813, 0.8776963668618512, 0.6533362985380224, 0.5258295402822944]

Сравнение результатов:
Разница между ручным МНК и llsq: 1.057
Разница между ручным МНК и GLM: 0.0
Матрица X2 = [1 X] позволяет оценить модель y = β₀ + β₁x₁ + β₂x₂ + β₃x₃.
Теоретически, МНК-оценка находится как a = (X2'X2)⁻¹X2'y.
Это стандартный метод решения overdetermined системы уравнений.
```

Рис. 4.15: Сопоставление результатов.

Далее нашла линию регрессии, используя данные (X, y). Построила график (X, y) с помощью точечного графика `scatter plot`. Добавила линию регрессии, используя функцию `abline!`. Также я добавила заголовок «График регрессии» и подписала оси x и y (рис. 4.16):

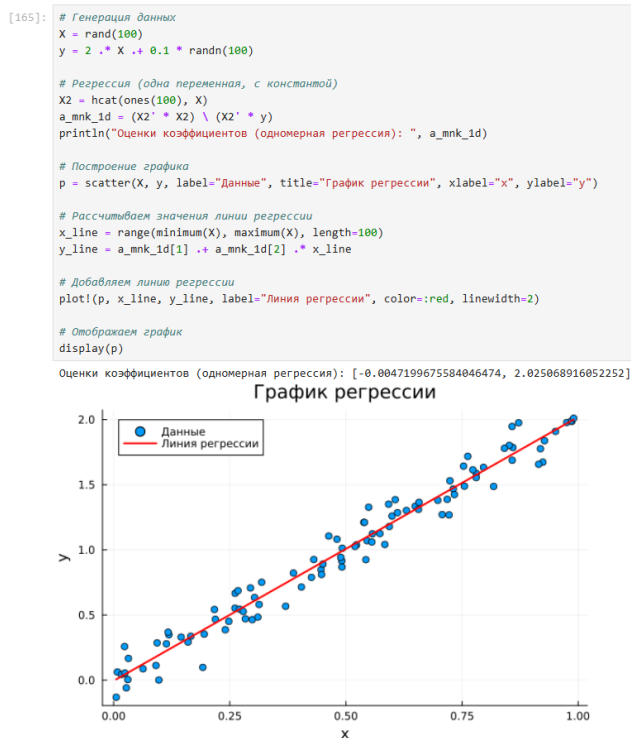


Рис. 4.16: Линейная регрессия и график.

4.2.3 Задание 3

Построим траекторию возможных цен на акции:

- S – начальная цена акции;
- T – длина биномиального дерева в годах;
- n – количество периодов;
- $h = T/n$ – длина одного периода;
- σ – волатильность акции;
- r – годовая процентная ставка;
- $u = \exp(rh + \sigma\sqrt{h})$;
- $d = \exp(rh - \sigma\sqrt{h})$;
- $p^* = \frac{\exp(rh) - d}{u - d}$;

Пусть $S = 100$, $T = 1$, $n = 10000$, $\sigma = 0.3$ и $r = 0.08$. Попробуем построить траекторию курса акций.

Построила модель ценообразования биномиальных опционов (рис. 4.17):

```
[168]: function createPath(S::Float64, r::Float64, sigma::Float64, T::Float64, n::Int64)
    h = T / n
    u = exp(r * h + sigma * sqrt(h))
    d = exp(r * h - sigma * sqrt(h))
    p_star = (exp(r * h) - d) / (u - d)

    # Генерируем траекторию
    path = Vector{Float64}(undef, n+1)
    path[1] = S

    for i in 2:(n+1)
        if rand() < p_star
            path[i] = path[i-1] * u
        else
            path[i] = path[i-1] * d
        end
    end

    return path
end

# Параметры
S = 100.0
T = 1.0
n = 10000
sigma = 0.3
r = 0.08;
```

Рис. 4.17: Модель ценообразования биномиальных опционов.

Построила график траектории цены акции (рис. 4.18):

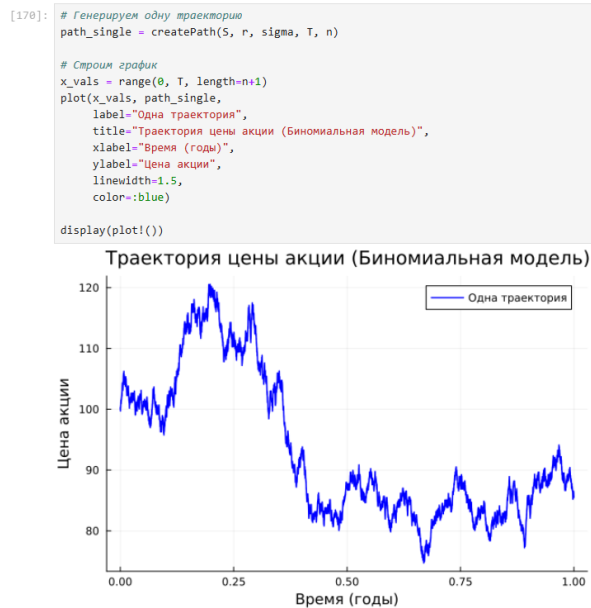


Рис. 4.18: График траекторий цены акции.

Создала функцию createPath ($S :: \text{Float64}$, $r :: \text{Float64}$, $\sigma :: \text{Float64}$, $T :: \text{Float64}$, $n :: \text{Int64}$), которая создает траекторию цены акции с учетом начальных параметров. С помощью createPath создала 10 разных траекторий и построила их все на одном графике (рис. 4.19-4.20):

```
[172]: # Генерируем 10 траекторий
num_paths = 10
paths = [createPath(S, r, sigma, T, n) for _ in 1:num_paths]

# Строим все траектории на одной графике
p = plot(title="10 траекторий цены акции", xlabel="Время (годы)", ylabel="Цена акции", legend=false)

for path in paths
    plot!(p, x_vals, path, linewidth=0.8, alpha=0.7)
end

display(p)
```

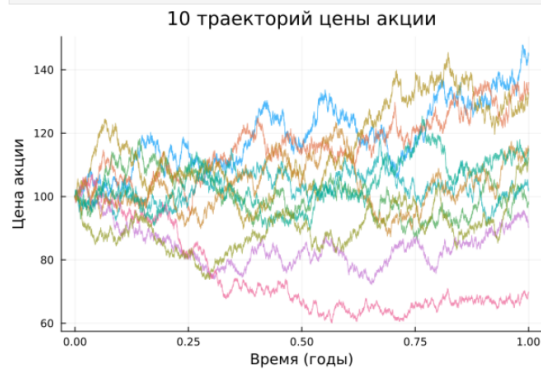


Рис. 4.19: График для различных траекторий цены.

```
[174]: paths_parallel = Vector{Vector{Float64}}(undef, num_paths)

Threads.@threads for i in 1:num_paths
    paths_parallel[i] = createPath(S, r, sigma, T, n)
end

# Строим график
p_parallel = plot(title="10 траекторий (параллельно)", xlabel="Время (годы)", ylabel="Цена акции", legend=false)

for path in paths_parallel
    plot!(p_parallel, x_vals, path, linewidth=0.8, alpha=0.7)
end

display(p_parallel)
```

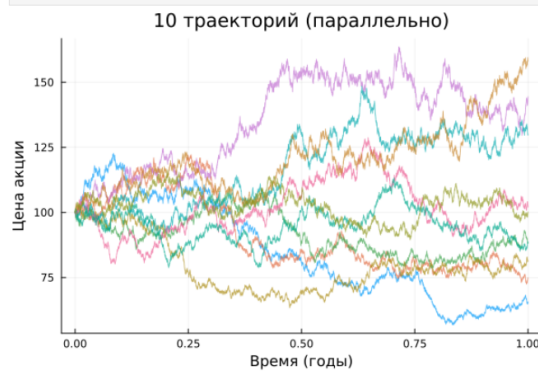


Рис. 4.20: Параллельное вычисление траекторий.

5 Выводы

Освоила специализированные пакеты Julia для обработки данных.